> **Facilitator answer key — Holocron capstone worked solution.** Facilitator-only — lives in the facilitator folder of every repo; do not hand to participants. Part of the internally-consistent set in this folder (see `README.md`).

AI Spec — Holocron (release-1 slice)

Sibling to: PRD-light (`prd-light.md`), TCD-light (`tcd-light.md`), TMD-light (`tmd-light.md`), SEP-light (`sep-light.md`), DP-light (`dp-light.md`) **Authors:** Holocron capstone quad (model solution) | **Status:** Reviewed **AI Provenance:** AI-assisted draft; every section validated against a source artifact — see the log in Section 8.

This is the integrated, engineering-ready spec. It is deliberately *shorter* than the sum of PRD + TCD + TMD: the synthesis an engineer reads first, with citations back to the deeper artifacts. Every claim traces to the PRD/TCD/TMD.

1. Headline

Holocron gives content owners across 150+ countries a single governed place to create, review, and publish UI strings — so 20–30+ consuming apps fetch localized text at render time (≤200ms p95) without hardcoding strings or waiting on a redeploy.

2. Engineering-ready summary

Problem and users (PRD §1–3). UI text today is hardcoded and shipped inside each app, so any wording, tone, or localization change requires an engineering change and a redeploy across 20–30+ apps. Holocron moves UI strings out of code into a centralized, governed store. The users are *content owners* (create/edit/publish source `en-US` strings and manage translation variants), *reviewers* (request/complete review before publish),

and *consuming apps* (read published strings at render time). Scale target: ~500,000 strings, 30–40 languages, `en-US` as source of truth, Internal data classification.

Release-1 scope (PRD §4–5). IN: access roles, product/application setup + namespace, create/edit/publish source strings, immutable version history + audit, search, delivery to consuming apps, request + complete review, and translation variants with their lifecycle. OUT (defensible non-goals): aliases, advanced reviewer configuration, export/import, rollback, AI-assisted translation, Figma integration, per-locale screenshot preview. An engineer should build only the IN list; the OUT list is explicitly deferred, not forgotten.

Architecture stance (TCD §1). Holocron is a **modular service on AKS** — one deployable with internally separated modules, not a microservice mesh and not an unscalable monolith. The decisive design move is **two physically separate paths over one PostgreSQL system of record**: a low-traffic **authoring/management path** (create/edit/publish/review; management app at 99.5%) and a high-traffic, read-optimized **delivery path** fronted by **Azure Cache for Redis** and served through **API Management + Application Gateway** (delivery at 99.9%, p95 ≤ 200ms). Rationale: 20–30+ apps fetching on every render generate orders of magnitude more traffic than the handful of editors; collapsing both onto one path would let read spikes starve publishing and force scaling the expensive audit machinery just to serve cache-friendly reads. The publish + audit-write transaction stays inside one process; a service is extracted only when a module's scale or ownership demands it.

Data and delivery (TMD §1, §3). The system of record is Azure Database for PostgreSQL. Core entities are immutable-by-design: a namespaced **Source String** key never changes, each edit creates a new immutable **Source String Version**, and the same versioning applies to **Translation Variants**. Publishing records a **Publication Selection** (approved | previous | `en-US` fallback), and every mutating operation emits an immutable **AuditEntry**. Consuming apps read through a single delivery endpoint — `GET /v1/products/{productId}/namespaces/{namespace}?locale=…` — authenticated with a Microsoft Entra ID token, returning the published strings for that namespace/locale with an explicit `fallback` flag when a translation is missing and `en-US` is served instead.

How an engineer scopes this. Build the authoring CRUD + review flow against Postgres with an append-only audit stream (Event Hubs → Blob archive); build the delivery read path as a separate horizontally scaled module that reads through Redis (read-through + backfill on miss) and degrades to direct Postgres reads if Redis is down. Publishing invalidates/refreshes the cache so the change propagates to delivery within ≤60s. Identity, RBAC, and rate limiting are enforced at the API Management edge and re-checked server-side per operation.

3. Data + API contract

Entities (TMD §1). All immutable rows are append-only; "edit" means "new version."

Entity	Key facts
Product / Application	Top-level tenant; owns namespaces and product-scoped roles.
Source String	**Immutable namespaced key** (`product.namespace.key`); source locale is `en-US` .
Source String Version	Immutable; one row per edit; carries the source text + author + timestamp.
Translation Variant	A locale rendering of a Source String; created/managed by content owners.
Translation Version	Immutable; one row per translation edit; has a lifecycle (draft → reviewed → published).
Review Request	Links a version to a reviewer; states request → complete; optional by default.
Publication Selection	Per locale at publish: `approved | previous | en-US fallback` .
AuditEntry	**Immutable**, one per mutating op; actor from Entra ID token; streamed to Event Hubs → Blob.
User	Microsoft Entra ID identity, role-scoped (viewer / editor / reviewer / admin), product-scoped.

Delivery endpoint (TMD §3) — the read contract 20–30+ apps depend on.

```
GET /v1/products/{productId}/namespaces/{namespace}?
locale=<BCP-47>
Authorization: Bearer <Entra ID token>

200 OK    → [ { "key": "...", "value": "...",
"locale": "...",
               "version": 42, "fallback": false },
... ]
401       → missing/invalid token
403       → token valid but not authorized for this
product
404       → product or namespace not found
429       → per-consumer rate limit exceeded (50
req/s, burst 100)
```

`fallback: true` means the requested `locale` had no published translation and `en-US` was served in its place — the consumer must be able to render that transparently.

Authoring endpoints (TMD §3) — the write side, RBAC-gated.

```
POST
/v1/products/{productId}/namespaces/{namespace}/string
     → create a Source String (+ first Source String
Version)   [role: editor]

POST /v1/.../strings/{key}/versions
     → add a Source String Version (edit = new
immutable version) [role: editor]

POST /v1/.../strings/{key}/review-requests
     → request review of a version
[role: editor]
PATCH /v1/.../review-requests/{id}  { "state":
"complete" }
     → complete a review
[role: reviewer]

POST /v1/.../strings/{key}/publish
     { "locale": "...", "selection":
"approved|previous|en-US" }
     → publish; writes Publication Selection +
AuditEntry in one tx.
       Publishing without a completed review is
allowed by default
       but records an audited publish-time override.
[role: editor/admin]
```

4. Sequence + failure handling

Publish → invalidate → deliver, held to the delivery p95 ≤ 200ms / propagation ≤ 60s SLOs (TMD §4):

```
HAPPY — publish reaches a consumer
  1. Editor POST /publish (approved selection)
  2. Postgres: write Publication Selection +
AuditEntry in ONE transaction  (commit = source of
truth)
  3. Emit AuditEntry to Event Hubs (async; never
blocks the commit)
  4. Invalidate/refresh Redis for {product,
namespace, locale}
  5. Consumer GET /namespaces/{ns}?locale=fr-FR  →
Entra token OK at APIM edge
  6. Redis HIT → return [{..., fallback:false}]
p95 ≤ 200ms
  ── published change visible to apps within ≤ 60s
(propagation SLO) ──

SAD — missing translation
  5'. Consumer requests locale=fr-FR, no published
fr-FR translation
  6'. Serve en-US value with fallback:true  → 200 OK
(never a 404 for a live key)

SAD — cache miss / Redis down
  6a. Redis MISS → read-through from Postgres,
backfill Redis, return 200
  6b. Redis DOWN → delivery degrades to direct
Postgres reads
                    (higher latency, SLO-protected by
the 50 req/s rate limit)

SAD — auth / abuse
  401 invalid token · 403 wrong product scope · 404
unknown product/namespace
  429 over 50 req/s (burst 100) → consumer holds
last good client-side cache

WEIRD — audit stream unavailable
  Event Hubs down → buffer + retry with
backpressure; the Postgres publish
  transaction still commits (audit is append-only
downstream, never blocks the write).
```

5. Constraints

Performance / availability SLOs (TCD §3, §4) — use exact figures.

Constraint	Target
Delivery fetch latency	p95 ≤ 200ms / p99 ≤ 500ms
Delivery availability	99.9%
Management app availability	99.5%
Publish → delivery propagation	≤ 60s
Per-consumer rate limit	50 req/s, burst 100

Security (TCD §3, §4; PRD §7). Microsoft Entra ID token validation on every management and delivery call, at the API Management edge and re-checked server-side. RBAC is enforced server-side per operation (product-scoped roles; least-privilege delivery identity), never trusted from the client. Every mutating op writes an **immutable AuditEntry** (actor from the token) → Event Hubs → Blob archive; publish-time review overrides are themselves audited. Secrets (connection strings, cache keys, signing material) live in Azure Key Vault; auth fails closed (cached JWKS validates existing tokens, new sign-ins blocked during a token-endpoint outage).

Compliance. Internal data classification. Immutable, append-only audit retained per Internal policy in Blob Storage; the audit trail — not enforced pre-publish gates — is the compliance control for the optional-review default (see §6, decision 2).

Scale. ~500,000 strings, 30–40 languages, 150+ countries, 20–30+ consuming apps. Delivery pods scale horizontally on read QPS and survive a management-app deploy or outage by serving from Redis + a Postgres read replica.

6. Decisions made (and not made)

Made (TCD §5):

1. **Redis read-cache in front of delivery**, accepting ≤60s publish→delivery staleness. Revisit if a consumer has a hard <5s propagation requirement (e.g., a legal/pricing string that must flip instantly).

2. **Reviews optional by default + audited publish-time override** (unless a review designation is set), rather than mandatory gates on every publish. Accepted cost: a string can ship unreviewed; we rely on audit, not prevention. Revisit if governance/compliance mandates enforced approval for a product or locale class.
3. **Immutable namespaced keys** (and immutable versions). Accepted cost: renaming means create-new + deprecate-old. Revisit if in-place renames become unmanageable at scale.
4. **Missing translation → `en-US` fallback with an explicit `fallback` indicator**, so a live key never 404s and consumers can render the fallback transparently.

Not decided (open — do not invent answers):

- **RPO / RTO targets** for the Postgres system of record and audit archive — deferred, pending Platform/SRE.
- **Alias governance** — aliases are out of the release-1 slice; their governance model is unspecified.
- **Export/import format** — out of slice; exchange format is an open question.

7. Stakeholders + sign-off

From SEP §1 + TCD §6.

Stakeholder	Interest	Status
Anna Woods — Product (Holocron)	Scope slice + release-1 value	Approved
Nick Grant — Engineering / delivery	AKS topology, delivery SLOs, Redis/Postgres split	Approved
Enterprise Infrastructure	Entra ID scoping + AKS platform	Approved
Legal / Compliance	Audit retention + review gates	Conditional — revisit if compliance mandates enforced pre-publish approval
Architecture	Immutable namespaced key schema	Approved (pending Key Vault rotation runbook)

8. Provenance log

AI-drafted spec; human judgment shaped every section. Each row records what AI assisted with and how it was validated against a source artifact — the discipline this course teaches. **A spec that ships without this section ships fiction.**

#	Section	AI-assisted with	Validation	St
1	§1 Headline	Compress what/who/outcome into one sentence	Checked scale + p95 figure against PRD §1–3 and TCD §3; user roles match PRD §2–3	✅ Va
2	§2 Engineering-ready summary	Synthesize problem/scope/stance/data into 5 paragraphs	Each paragraph traced to PRD §1–5, TCD §1, TMD §1/§3; IN/OUT list checked against PRD §4–5 verbatim	✅ Va
3	§3 Data + API contract	Draft entity table + endpoint signatures	Entities + immutability rules checked against TMD §1; delivery endpoint, status codes, `fallback` flag checked against TMD §3; RBAC roles checked against PRD §2	✅ Va
4	§4 Sequence + failure handling	Draft happy/sad/weird publish→deliver flow	Checked against TCD §2 integration failure handling + TMD §4; SLO figures (≤200ms, ≤60s, 50 req/s) match	✅ Va

#	Section	AI-assisted with	Validation	St
			TCD §3/§4 exactly	
5	§5 Constraints	Assemble SLO + security + compliance + scale table	Every SLO figure copied from TCD §3/§4 (not paraphrased); security/audit claims traced to TCD §3–4 STRIDE + PRD §7	✅ Va
6	§6 Decisions made (and not made)	List 4 trade-offs + open items	4 decisions match TCD §5 trade-off table (choice + accepted cost + revisit trigger); open items (RPO/RTO, aliases, export) confirmed as NOT-decided in the spine, not invented	✅ Va
7	§7 Stakeholders + sign-off	Merge named stakeholders + status	Names from SEP §1 spine; statuses reconciled with TCD §6 sign-off table (Legal + Architecture conditions preserved)	✅ Va
8	§8 Provenance log	Draft this table	Self-referential; instructor confirms	✅ Va

#	Section	AI-assisted with	Validation	St
			every other row cites a real source section before this spec is called "Reviewed"	